

Read Isolation, Consistency, and Recency

Isolation Guarantees

Read Uncommitted

Depending on the read concern, clients can see the results of writes before the writes are durable:

- Regardless of a write's `write concern`, other clients using `"local"` or `"available"` read concern can see the result of a write operation before the write operation is acknowledged to the issuing client.
- Clients using `"local"` or `"available"` read concern can read data which may be subsequently rolled back during replica set failovers.

For operations in a multi-document transaction, when a transaction commits, all data changes made in the transaction are saved and visible outside the transaction. That is, a transaction will not commit some of its changes while rolling back others.

Until a transaction commits, the data changes made in the transaction are not visible outside the transaction.

However, when a transaction writes to multiple shards, not all outside read operations need to wait for the result of the committed transaction to be visible across the shards. For example, if a transaction is committed and write 1 is visible on shard A but write 2 is not yet visible on shard B, an outside read at read concern `"local"` can read the results of write 1 without seeing write 2.

Read uncommitted is the default isolation level and applies to `mongod` standalone instances as well as to replica sets and sharded clusters.

Read Uncommitted And Single Document Atomicity

Write operations are atomic with respect to a single document; i.e. if a write is updating multiple fields in the document, a read operation will never see the document with only some of the fields updated.

However, although a client may not see a *partially* updated document, read uncommitted means that concurrent read operations may still see the updated document before the changes are made durable.

With a standalone `mongod` instance, a set of read and write operations to a single document is serializable. With a replica set, a set of read and write operations to a single document is serializable *only* in the absence of a rollback.

Read Uncommitted And Multiple Document Write

When a single write operation (e.g. `db.collection.updateMany()`) modifies multiple documents, the modification of each document is atomic, but the operation as a whole is not atomic.

When performing multi-document write operations, whether through a single write operation or multiple write operations, other operations may interleave.

For situations that require atomicity of reads and writes to multiple documents (in a single or multiple collections), MongoDB supports distributed transactions, including transactions on replica sets and sharded clusters.

For more information, see [transactions](#)

IMPORTANT

In most cases, a distributed transaction incurs a greater performance cost over single document writes, and the availability of distributed transactions should not be a replacement for effective schema design. For many scenarios, the denormalized data model (embedded documents and arrays) will continue to be optimal for your data and use cases. That is, for many scenarios, modeling your data appropriately will minimize the need for distributed transactions.

For additional transactions usage considerations (such as runtime limit and oplog size limit), see also [Production Considerations](#).

Without isolating the multi-document write operations, MongoDB exhibits the following behavior:

1. Non-point-in-time read operations. Suppose a read operation begins at time t_1 and starts reading documents. A write operation then commits an update to one of the documents at some later time t_2 . The reader may see the updated version of the document, and therefore does not see a point-in-time snapshot of the data.
2. Non-serializable operations. Suppose a read operation reads a document d_1 at time t_1 and a write operation updates d_1 at some later time t_3 . This introduces a read-write dependency such that, if the operations were to be serialized, the read operation must precede the write operation. But also suppose that the write operation updates document d_2 at time t_2 and the read operation subsequently reads d_2 at some later time t_4 . This introduces a write-read dependency which would instead require the read operation to come *after* the write operation in a serializable schedule. There is a dependency cycle which makes serializability impossible.
3. Reads may miss matching documents that are updated during the course of the read operation.

Cursor Snapshot

MongoDB cursors can return the same document more than once in some situations. As a cursor returns documents, other operations may interleave with the query. If one of these operations changes the indexed field on the index used by the query, then the cursor could return the same document more than once.

Queries that use unique indexes can, in some cases, return duplicate values. If a cursor using a unique index interleaves with a delete and insert of documents sharing the same unique value, the cursor may return the same unique value twice from different documents.

Consider using read isolation. To learn more, see Read Concern `"snapshot"`.

Monotonic Writes

MongoDB provides monotonic write guarantees, by default, for standalone `mongod` instances and replica set.

For monotonic writes and sharded clusters, see Causal Consistency.

Real Time Order

For read and write operations on the primary, issuing read operations with `"linearizable"` read concern and write operations with `"majority"` write concern enables multiple threads to perform reads and writes on a single document as if a single thread performed these operations in real time; that is, the corresponding schedule for these reads and writes is considered linearizable.

TIP

See also:

Causal Consistency

Causal Consistency

If an operation logically depends on a preceding operation, there is a causal relationship between the operations. For example, a write operation that deletes all documents based on a specified condition and a subsequent read operation that verifies the delete operation have a causal relationship.

With causally consistent sessions, MongoDB executes causal operations in an order that respect their causal relationships, and clients observe results that are consistent with the causal relationships.

Client Sessions and Causal Consistency Guarantees

To provide causal consistency, MongoDB enables causal consistency in client sessions. A causally consistent session denotes that the associated sequence of read operations with `"majority"` read concern and write operations with `"majority"` write concern have a causal relationship that is reflected by their ordering. **Applications must ensure that only one thread at a time executes these operations in a client session.**

For causally related operations:

1. A client starts a client session.

IMPORTANT

Client sessions only guarantee causal consistency for:

- Read operations with `"majority"`; i.e. the return data has been acknowledged by a majority of the replica set members and is durable.

- Write operations with "majority" write concern; i.e. the write operations that request acknowledgment that the operation has been applied to a majority of the replica set's voting members.

For more information on causal consistency and various read and write concerns, see [Causal Consistency and Read and Write Concerns](#).

2. As the client issues a sequence of read with "majority" read concern and write operations (with "majority" write concern), the client includes the session information with each operation.
3. For each read operation with "majority" read concern and write operation with "majority" write concern associated with the session, MongoDB returns the operation time and the cluster time, even if the operation errors. The client session keeps track of the operation time and the cluster time.

NOTE

MongoDB does not return the operation time and the cluster time for *unacknowledged* (w: 0) write operations. Unacknowledged writes do not imply any causal relationship.

Although, MongoDB returns the operation time and the cluster time for read operations and acknowledged write operations in a client session, only the read operations with "majority" read concern and write operations with "majority" write concern can guarantee causal consistency. For details, see [Causal Consistency and Read and Write Concerns](#).

4. The associated client session tracks these two time fields.

NOTE

Operations can be causally consistent across different sessions. MongoDB drivers and `mongosh` provide the methods to advance the operation time and the cluster time for a client session. So, a client can advance the cluster time and the operation time of one client session to be consistent with the operations of another client session.

Causal Consistency Guarantees

The following table lists the causal consistency guarantees provided by causally consistent sessions for read operations with "majority" read concern and write operations with "majority" write concern.

Guarantees	Description
Read your writes	Read operations reflect the results of write operations that precede them.
Monotonic reads	Read operations do not return results that correspond to an earlier state of the data than a preceding read operation. For example, if in a session: • write 1 precedes write 2, • read 1 precedes read 2, and • read 1 returns results that reflect write 2

Guarantees	Description
	then read 2 cannot return results of write 1.
Monotonic writes	Write operations that must precede other writes are executed before those other writes. For example, if write 1 must precede write 2 in a session, the state of the data at the time of write 2 must reflect the state of the data post write 1. Other writes can interleave between write 1 and write 2, but write 2 cannot occur before write 1.
Writes follow reads	Write operations that must occur after read operations are executed after those read operations. That is, the state of the data at the time of the write must incorporate the state of the data of the preceding read operations.

Read Preference

These guarantees hold across all members of the MongoDB deployment. For example, if, in a causally consistent session, you issue a write with "majority" write concern followed by a read that reads from a secondary (i.e. read preference secondary) with "majority" read concern, the read operation will reflect the state of the database after the write operation.

Isolation

Operations within a causally consistent session are not isolated from operations outside the session. If a concurrent write operation interleaves between the session's write and read operations, the session's read operation may return results that reflect a write operation that occurred *after* the session's write operation.

MongoDB Drivers

TIP

Applications must ensure that only one thread at a time executes these operations in a client session.

Clients require MongoDB drivers updated for MongoDB 3.6 or later:

Java 3.6+	C# 2.5+	Perl 2.0+
Python 3.6+	Node 3.0+	PHPC 1.4+
C 1.9+	Ruby 2.5+	Scala 2.2+
Go 1.8+	Rust 2.1+	C++ 3.6.6+
	Swift 1.2+	

Examples

IMPORTANT

Causally consistent sessions can only guarantee causal consistency for reads with "majority" read concern and writes with "majority" write concern.

Consider a collection `items` that maintains the current and historical data for various items. Only the historical data has a non-null `end` date. If the `sku` value for an item changes, the document with the old `sku` value needs to be updated with the `end` date, after which the new document is inserted with the current `sku` value. The client can use a causally consistent session to ensure that the update occurs before the insert.

► Use the **Select your language** drop-down menu in the upper-right to set the language of this example.

```
C#
using (var session1 = client.StartSession(new ClientSessionOptions {
    CausalConsistency = true }))
{
    var currentDate = DateTime.UtcNow.Date;
    var items = client.GetDatabase(
        "test",
        new MongoDBDatabaseSettings
        {
            ReadConcern = ReadConcern.Majority,
            WriteConcern = new WriteConcern(
                WriteConcern.WMode.Majority,
                TimeSpan.FromMilliseconds(1000))
        })
        .GetCollection<BsonDocument>("items");
    items.UpdateOne(session1,
        Builders<BsonDocument>.Filter.And(
            Builders<BsonDocument>.Filter.Eq("sku", "111"),
            Builders<BsonDocument>.Filter.Eq("end", BsonNull.Value)),
        Builders<BsonDocument>.Update.Set("end", currentDate));
    items.InsertOne(session1, new BsonDocument
    {
        {"sku", "nuts-111"},
        {"name", "Pecans"},
        {"start", currentDate}
    });
}
```

If another client needs to read all current `sku` values, you can advance the cluster time and the operation time to that of the other session to ensure that this client is causally consistent with the other session and read after the two writes:

```
C#
using (var session2 = client.StartSession(new ClientSessionOptions {
    CausalConsistency = true }))
{
    session2.AdvanceClusterTime(session1.ClusterTime);
    session2.AdvanceOperationTime(session1.OperationTime);
    var items = client.GetDatabase(
        "test",
        new MongoDBDatabaseSettings
```

```

{
  ReadPreference = ReadPreference.Secondary,
  ReadConcern = ReadConcern.Majority,
  WriteConcern = new WriteConcern(WriteConcern.WMode.Majority,
  TimeSpan.FromMilliseconds(1000))
})
.GetCollection<BsonDocument>("items");
var filter = Builders<BsonDocument>.Filter.Eq("end", BsonNull.Value);
foreach (var item in items.Find(session2, filter).ToEnumerable())
{
  // process item
}
}

```

Limitations

The following operations that build in-memory structures are not causally consistent:

Operation	Notes
<code>collStats</code>	
<code>\$collStats</code> with <code>latencyStats</code> option.	
<code>\$currentOp</code>	Returns an error if the operation is associated with a causally consistent client session.
<code>createIndexes</code>	
<code>dbHash</code>	
<code>dbStats</code>	
<code>getMore</code>	Returns an error if the operation is associated with a causally consistent client session.
<code>\$indexStats</code>	
<code>mapReduce</code>	
<code>ping</code>	Returns an error if the operation is associated with a causally consistent client session.
<code>serverStatus</code>	Returns an error if the operation is associated with a causally consistent client session.
<code>validate</code>	

